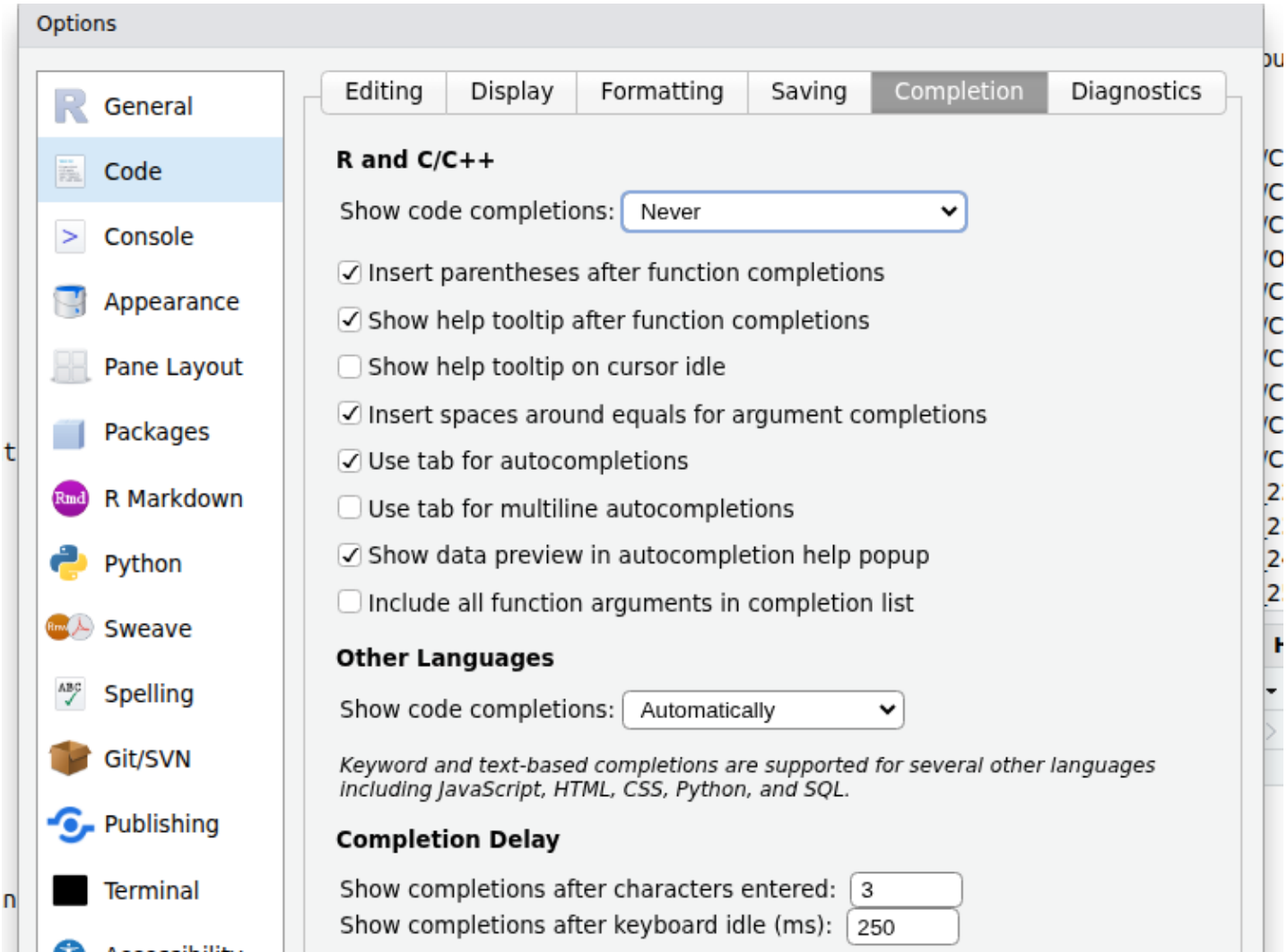
Lab 8: Speeding up R and R graphics

Matteo Fasiolo

Rstudio set up (ignore if you have already done this)

Rstudio has auto-suggestion features which we want to disable.

To do that go to `Tools` > `Global Options` > `Code` > `Completion` and set `Show code completions` to `Never`.



Homework 1: Practicing rowSums

The function `rowSums` sums over each row of a matrix.

```
( A <- matrix(c(1,2,3,4,5,6), 2, 3) )
```

```
     [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
```

```
rowSums(A)
```

```
[1]  9 12
```

```
c(sum(A[1, ]), sum(A[2, ]))
```

```
[1]  9 12
```

Read the manual of `rowSums` and play with a few more examples.

There is a similar function called `colSums` that performs column sums. Try it yourself.

Homework 2: Pairwise Distance Calculation

Assume we have 2 matrices:

- $\mathbf{A}$ of dimension (500×784);
- $\mathbf{B}$ of dimension (784×800).

Objective: computing a (500×800) matrix $\mathbf{D}$ where:

$$D[i, j] = \text{Euclidean_Distance}(A[i,], B[, j]).$$

Steps:

1. Create two matrices of the correct size filled with random observations from a standard normal distribution. To do this, use `rnorm(n)` which outputs a vector of size `n` filled with samples from a standard normal distribution.

Homework 2a: Scalar Computation

2. Write a function called `pdist1` that
   - takes two matrices A, B as inputs
   - outputs a matrix D, where $D_{i,j} = \text{Euc_Dist}(A_{[i,]}, B_{[,j]})$.
   - Your code should have **3 nested for loops**, i.e., you should write non-vectorized code.
   - Test the function on matrices generated in step 1.

Homework 2b: Vectorization

3. Write another function called `pdist2`, that does exactly the same thing, using only **two for loops**.
   - Hint, consider the vectorised `dist` function from last week.
4. Write a function `pdist3`, using **only one for loop**.
   - Hint, you can compute the whole i-th row in D by using:

     $$||\mathbf{x}-\mathbf{y}|| = ((\mathbf{x}-\mathbf{y})^T(\mathbf{x}-\mathbf{y}))^{1/2} = (\mathbf{x}^T\mathbf{x}-2\mathbf{x}^T\mathbf{y}+\mathbf{y}^T\mathbf{y})^{1/2}.$$

   - If $\mathbf{x} = A[i,]$ and $\mathbf{y} = B[,j]$, you can compute

     $$D[i,] = \{||A[i,]-B[,1]||, ||A[i,]-B[,2]||, \cdots, ||A[i,]-B[,800]||\}$$

     without any `for` loop (hence only the loop over i is left).

Homework 2c: Vectorization

5. Write another function called `pdist4`, that does exactly the same thing **without using any for loop**.
   - You should be able to do this using only `sum`, `t()` and `rowSums/colSums`.
   - Express D by matrix algebra using A and B.
   - $D = \sqrt{a1_b^\top + 1_a b^\top - 2AB}$, where
     - `a := rowSums(A^2)`, `b := colSums(B^2)`
     - 1_a is a column vector with `nrow(A)` elements and each element equal to 1
     - 1_b is similar but has `ncol(B)` elements.
6. Test all above functions using the matrices you generated in the first step:
   - How long does it take them to run?
   - Do you get the correct result?

Homework 3: Animated Visualization

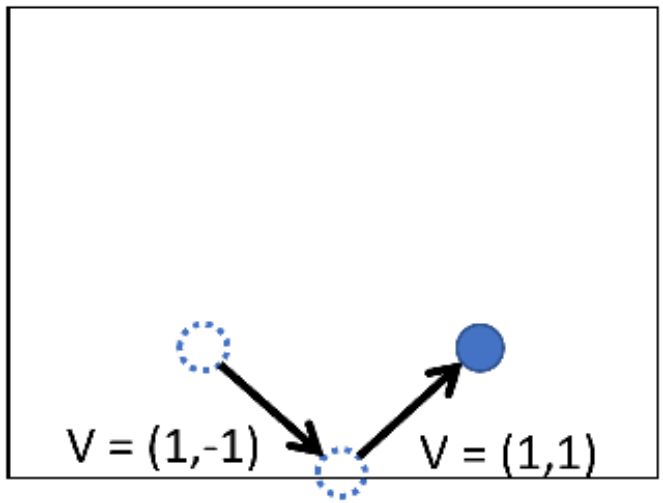
This exercise is inspired by a scene in "The Office":

https://www.youtube.com/watch?v=QOtuX0jL85Y

We are going to write an animated visualization that mimics the DVD logo bouncing around the edge of the TV.

1. Create an empty plot, whose x-axis and y-axis range from -5 to 5.
2. Initialize two vectors:
   - $x = (0, 0)$ and $v = (0.23456, 0.12345)$.
   - Draw the vector x as a **red** dot in the plot.
   - Add plot title: "point simulation, press ESC to stop."
     - Hint: `?title`

3. Now let us write a function that updates the point's `x` location based on the velocity vector `v`.
    - If the position of `x` is slightly out of the box, the point will be **bounced back**.



4. Write a function `update(x,v)` that takes two vectors `x` and `v` as inputs and returns the updated `x` and `v`.
    - First, the function checks if `x` is outside of the boundary.
    - If so, it updates the velocity vector by "reflecting it". See the picture in the previous slide.
    - Then, update `x <- x + v`
    - Finally, `return( list(x,v) )`
    - The `list` combines two vectors and allows you to return multiple values from a function.
    - Hint: `x[2] > 5 || x[2] < -5` is true if `x` is above the ceiling or is below the floor.

Note on lists, if you do `z <- list(x,v)` then `z[[1]]` returns `x` and `z[[2]]` returns `v`.

That's all you need to know, but if you prefer you can use a matrix to store `x` and `v`.

5. Your code should look like this:

```
update <- function(x,v){
  # TODO: Check the boundary collision,
  #       and update v and x.
  return(list(x,v))
}

x <- c(0,0) #the initial position of x
v <- c(0.23456,0.12345) #the initial velocity of x

while(T){
  # TODO: create a new plot and draw x's current
  #       location
  x <- xv[[1]]
  v <- xv[[2]]
  Sys.sleep(0.1)  # wait a bit to give Rstudio
                  # the time to plot
}
```

Your plot should look like this:

https://github.com/mfasiolo/MATH10017_26/blob/master/Week_by_week/Week_8/Lab/lab8.mp4

(you need to download the video to watch it)

In the TV show, each time the logo bounces, it changes color randomly. Could you revise your code so that the point in your plot also changes its color every time it bounces?

Submission

Submit your completed `vectorisation.R` and `animation.R` files from Homework 2 and 3.

Before submitting, make sure your code runs.